Learning-Driven Code Evolution and Multi-Objective Heuristic Optimization for the OpenROAD Physical Design Engine

Research Progress Report: Architecture, Methodology, and Current Empirical Results

Kavya Kumar Agrawal
Department of Electronics and Communication Engineering
Indian Institute of Technology (BHU), Varanasi
agrawalk1028@gmail.com

November 4, 2026

Abstract

Physical design tools in VLSI synthesis rely heavily on expert-crafted greedy heuristics for clock tree synthesis (CTS) and timing-driven gate resizing. While these algorithms are computationally tractable, their static thresholds often yield suboptimal trade-offs across diverse circuit topologies. In this work, we present a learning-driven evolutionary framework that automatically optimizes critical heuristic kernels in the open-source OpenROAD physical design engine. We target two central subsystems: TritonCTS (`src/cts`) and Resizer (`src/rsz`). Across an empirical evaluation of 3,500 candidate modifications and 61 applied structural blocks, the evolved code variants demonstrate a mean improvement of +1.15% in Worst Negative Slack (WNS) and +1.15% in Total Negative Slack (TNS) with top-tier candidates achieving up to +4.99% WNS and +5.00% TNS gains, alongside a mean wall-clock runtime reduction of 0.251 seconds. This report details the architectural transformations, the multi-objective optimization formulation, the invariant preservation defense against semantic divergence, and the roadmap for full-flow ASIC benchmark evaluation.

1 Introduction and Problem Statement

Modern VLSI physical design involves solving sequentially coupled, highly constrained NP-hard optimization problems. Stages such as Clock Tree Synthesis (CTS) and Timing/Power Optimization (Resizer) in OpenROAD employ deterministic heuristics parameterized by static numerical weights and hard-coded thresholds. For instance, in TritonCTS, sink clustering is guided by fixed geometric radius constraints and uniform capacity limits, while in Resizer, gate sizing and buffer insertion rely on fixed slack thresholds to balance setup, hold, and dynamic power.

Because optimal heuristic parameters vary drastically between register-dense arithmetic pipelines and loosely coupled control logic, static heuristics leave substantial timing and runtime margins on the table. We formulate the automated refactoring of these kernels as a learning-driven code evolution problem. The objective is to discover algorithmic policy adaptations and code transformations in OpenROAD that systematically improve timing metrics while preserving strict functional and physical design invariants.

2 Codebase Architecture and Structural Transformations

The modifications focus on two core subsystems in the OpenROAD source tree, comprising 23 translation units:

- **TritonCTS (`src/cts`)**: Responsible for balanced clock tree construction. Target files include:
  - `Clock.cpp`, `Clock.h`: Clock net representation and tree hierarchy management.
  - `Clustering.cpp`, `SinkClustering.cpp`: k-means and capacity-constrained geometric clustering of clock sinks.
  - `HTreeBuilder.cpp`, `TreeBuilder.cpp`: Top-level H-tree and bottom-up tree synthesis.
  - `LatencyBalancer.cpp`: Insertion delay matching across clock domains.
  - `TechChar.cpp`, `TechChar.h`: Technology characterization for buffer and wire segment delays.
- **Resizer (`src/rsz`)**: Responsible for post-placement timing and power closure. Target files include:

- `SetupLegacyBase.cc`: Critical path identification and setup slack recovery via iterative driver upsizing.
- `RepairHold.cc`: Hold violation mitigation through delay buffer insertion without degrading setup slack.
- `RecoverPower.cc`: Downsizing and V_t-swapping on positive-slack paths for power recovery.

2.1 Policy Parameterization and Dual-Branch Dispatch

To enable safe runtime exploration without destabilizing upstream pipelines, we introduced a centralized policy configuration abstraction:

1. `LearningPolicyConfig.h`: Encapsulates tunable heuristic hyperparameters, including:
   - Geometric clustering radius scaling factors (α_{cluster}).
   - Maximum sink fanout thresholds per clock driver.
   - Latency balancing tolerance windows (ϵ_{skew}).
   - Power recovery slack-backoff bounds (δ_{slack}).
2. `LearningMode.h`: Implements a thread-safe dispatch interface enabling runtime toggling between legacy baseline kernels and evolved policy kernels.
3. **Dual Source Implementations**: Each modified translation unit maintains explicit legacy (`*_old.cpp`) and evolved (`*_new.cpp`) counterparts, ensuring full binary compatibility and verifiable delta isolation.

3 Methodology and Optimization Approach

3.1 Multi-Objective Fitness Formulation

Each candidate code modification $c \in \mathcal{C}$ is evaluated through a multi-objective scalarized fitness function designed to simultaneously maximize timing closure and minimize execution overhead:

$$\mathcal{F}(c) = w_1 \cdot \Delta\text{WNS}_{\%}(c) + w_2 \cdot \Delta\text{TNS}_{\%}(c) - w_3 \cdot \Delta T_{\text{runtime}}(c) \tag{1}$$

where:

$$\Delta\text{WNS}_{\%}(c) = \frac{\text{WNS}(c) - \text{WNS}_{\text{base}}}{|\text{WNS}_{\text{base}}|} \times 100\%$$

$$\Delta\text{TNS}_{\%}(c) = \frac{\text{TNS}(c) - \text{TNS}_{\text{base}}}{|\text{TNS}_{\text{base}}|} \times 100\%$$

$$\Delta T_{\text{runtime}}(c) = T(c) - T_{\text{base}} \quad \text{(in seconds)}$$

In our empirical calibration, we set $w_1 = 0.45$, $w_2 = 0.45$, and $w_3 = 0.10$, heavily rewarding slack recovery while penalizing algorithmic regressions.

3.2 Correctness and Invariant Preservation

A critical question in source-to-source code optimization is: *How is semantic correctness guaranteed, given that unbounded C++ equivalence checking is undecidable and bounded equivalence is NP-hard?*

We address this fundamental challenge through a layered verification methodology:

1. **Structural Invariant Assertions**: Evolved kernels operate within bounded search spaces where all input/output physical constraints are checked:
   - *Clock Skew Bound*: Clock skew must satisfy $\text{Skew}(c) \leq \text{Skew}_{\max}$.
   - *Electrical DRCs*: Maximum transition time (slew) and maximum output capacitance rules must remain satisfied.
   - *Physical Placements*: Cell movement must be strictly confined to valid standard cell placement rows without overlapping macros or existing cells.
2. **Differential Oracle Testing**: The evolved candidate is executed concurrently against the legacy baseline on an identical timing graph. If any invariant violation, null-pointer dereference, or numeric divergence occurs, the candidate is discarded immediately.
3. **Deterministic Checksumming**: All generated patches, candidate source files, and dataset rows are bound by SHA-256 digests in `CHECKSUMS.json`, verified by an automated test harness covering 7,260 artifacts.

4 Empirical Results and Performance Analysis

4.1 Statistical Distribution of the Candidate Pool

A total of 3,500 candidate variants were generated, executed, and analyzed across 61 applied code blocks. Table 1 reports the statistical distribution across all evaluated metrics.

4.2 Key Empirical Findings

1. **Consistent Timing Gains**: Across the candidate population, 75% of candidates achieved non-negative

Table 1: Empirical Metric Distributions across 3,500 Evaluated Candidates.

Metric	Mean	Median	95th %ile	Max / Min
WNS Impact (%)	+1.151	+0.718	+4.383	+4.993 (Max)
TNS Impact (%)	+1.150	+0.700	+4.400	+4.998 (Max)
Combined Timing (%)	+1.150	+0.626	+4.445	+4.974 (Max)
Runtime Delta (s)	−0.251	−0.258	+0.248	−0.800 (Min)
Overall Score	1.060	0.592	4.016	4.520 (Max)

WNS improvements, with the 90th percentile exceeding +2.42% and top-tier candidates reaching $\approx +4.99\%$.

2. **Simultaneous Runtime Reduction**: The mean runtime delta is negative (−0.251 seconds), demonstrating that smarter spatial pruning in sink clustering and reduced search overhead in gate sizing improve efficiency rather than adding computational drag.

3. **Sustained Correlation**: The correlation between WNS and TNS improvement is consistently positive, verifying that optimizing for the worst critical path does not penalize secondary timing paths across the clock domain.

4.3 Analysis of Top-Performing Candidates

Table 2 lists representative top-performing candidates identified by the evolutionary search, detailing their specific algorithmic target, line modifications, timing gains, and overall fitness.

5 Benchmark Suite for Full-Flow Validation

To validate these optimizations within complete RTL-to-GDSII execution runs, we have selected standard open-source benchmark designs spanning varied scale and architectural properties:

1. **Control & Cryptographic Cores**:
   - `ibex`: A 2-stage in-order 32-bit RISC-V RV32IMC processor core (balanced logic-to-register ratio).
   - `aes_cipher`: High-throughput 128-bit AES cryptographic engine with dense combinational S-box networks.
2. **Signal & Multimedia Processing**:
   - `jpeg_encoder`: Pipelined 2D Discrete Cosine Transform and quantization datapath.
3. **High-Complexity Out-of-Order / 64-bit Cores**:
   - `cva6` (*Ariane*): A 6-stage single-issue 64-bit RISC-V application core with MMU and cache subsystems.
   - `sparc_core`: Multi-pipeline OpenSPARC execution unit with deep register files.
4. **Process Design Kits (PDKs)**: Nangate45 open standard cell library and ASAP7 7 nm predictive FinFET library for advanced node verification.

6 Conclusion and Next Steps

This work demonstrates that automated, learning-guided code evolution can systematically uncover heuristic optimizations in production VLSI physical design tools. By formulating multi-objective timing-runtime objectives and enforcing strict invariant preservation, the evolved OpenROAD CTS and Resizer routines achieve significant slack recovery while accelerating execution time. Immediate next steps include running complete OpenROAD-flow-scripts macro-placement and routing pipelines across the benchmark suite, performing parasitic extraction with OpenRCX, and validating final signoff timing via OpenSTA.

References

[1] T.-W. Lin, C.-K. Cheng, A. B. Kahng, et al., "The OpenROAD Project: Open-Source RTL-to-GDSII," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 4, pp. 681–694, 2021.

[2] G. Flach, M. Fogaça, J. Lu, et al., "TritonCTS: Global Clock Tree Synthesis for OpenROAD," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019.

[3] A. B. Kahng, C. N. Sze, and Z. Wang, "High-Performance Timing and Slew Repair for Physical Design," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 24, no. 3, 2019.

[4] D. Vogel, S. Garg, and M. He, "Automated Program Evolution for Algorithmic Optimization," in *International Conference on Machine Learning (ICML)*, 2024.

Table 2: Representative Top-Performing Evolved Candidates from `highest_labeled_candidates.csv`.

Candidate ID	**Target Subsystem / File**	**Lines (±)**	**ΔWNS (%)**	**ΔTNS (%)**	**ΔRuntime (s)**	**Overall Score**
`2c6d4f82`	`src/cts/LearningPolicyConfig.h`	$+8 / -8$	$+4.930\%$	$+4.981\%$	$-0.598\,\mathrm{s}$	**4.520**
`dd409bdf`	`src/cts/LearningPolicyConfig.h`	$+10 / -10$	$+4.988\%$	$+4.959\%$	$-0.118\,\mathrm{s}$	**4.488**
`f1eaf650`	`src/cts/LearningPolicyConfig.h`	$+9 / -9$	$+4.981\%$	$+4.833\%$	$-0.586\,\mathrm{s}$	**4.475**
`561eb502`	`src/cts/LearningPolicyConfig.h`	$+10 / -10$	$+4.773\%$	$+4.986\%$	$-0.618\,\mathrm{s}$	**4.453**
`8e2e0ecab`	`src/cts/LearningPolicyConfig.h`	$+8 / -8$	$+4.808\%$	$+4.998\%$	$-0.099\,\mathrm{s}$	**4.422**
`90626ac8`	`src/cts/LearningPolicyConfig.h`	$+10 / -10$	$+4.756\%$	$+4.906\%$	$-0.410\,\mathrm{s}$	**4.389**
`bdd09df1`	`src/cts/LearningPolicyConfig.h`	$+9 / -9$	$+4.897\%$	$+4.676\%$	$-0.698\,\mathrm{s}$	**4.377**
`0b7ad0f8`	`src/cts/LearningPolicyConfig.h`	$+1 / -1$	$+4.762\%$	$+4.790\%$	$-0.752\,\mathrm{s}$	**4.374**